Analytic Underpinnings of the Symbolic Answer-Reduction Pipeline

4 September 2026

Abstract

The objects manipulated by answer reduction are finite descriptions of machines, circuits, formulas, polynomials, samplers, and verifiers. The Julia prototype realizes part of the corresponding analytic chain as typed transformations of explicit intermediate representations. It constructs finite-field polynomials and conditionally linear samplers, and it checks coefficient identities, degree accounts, dependency blocks, marginal laws, and finite-instance histograms. The operator-algebraic and compression theorems remain imported. This note makes the description-level Turing-machine–lambda-calculus correspondence precise, identifies the fixed point, and records the evidence boundary at every rung.

Contents

1 The paper's machine model and its intensional obligations

1.1 Machines, inputs, time, and descriptions

We first repeat the conventions on which the later change of description language depends. This is necessary because the change is intensional: it must preserve programs, bounds, and programs which inspect other programs, not only partial functions.

Definition 1.1 (The paper's Turing-machine convention)**.** A k-input Turing machine has k input tapes, one work tape, and one output tape. Tapes are one-way infinite, indexed by $\mathbb{N}$, and use $\{0, 1, \sqcup\}$. Work and output tapes are initially blank. If the machine halts, its output is the longest nonblank initial string on the output tape, with the all-blank output interpreted as the one-bit string 0. On $x = (x_1, \ldots, x_k)$, write $M(x)$ for this string and write $M(x) = \perp$ if it never halts. Its instance time $\mathrm{TIME}_{M,x} \in \mathbb{N} \cup \{\infty\}$ is the number of transitions to the halt state.

Every machine has canonical binary code $\overline{M}$. The code lists its finite states and transition table using a fixed reasonable encoding; its bit length is $|\overline{M}|$, abbreviated $|M|$. For a bit string α, $[\alpha]_k$ is the k-input machine described by α (with the fixed convention for malformed strings). Tuples are encoded by enc_k: each data bit is replaced by $0 \mapsto 01$, $1 \mapsto 10$, and each component ends in 00. Thus $|\,\mathrm{enc}_k(x)| = 2\sum_i |x_i| + 2k$, and encoding takes linear time.

This is the model in the Turing-machine subsection of the paper. [1] Its universal-machine convention is quantitative. For each k there is a fixed one-tape U_k, and universal constants $C_U, c_U \geq 1$, such that if $[\alpha]_k(x)$ halts in T steps then $U_k(\mathrm{enc}_{k+1}(\alpha, x))$ produces the same answer within

$$C_U(k|\alpha||x|T)^{c_U}, \qquad |x| := \sum_i |x_i|.$$

The paper also assumes that a high-level machine which invokes a fixed submachine has description length linear in the embedded submachine code. Hardwiring a string z into one input of M is an effective source transformation $s(M, z)$; its output description has length $O(|M|+|z|+1)$, is produced in time $O(|M| + |z| + 1)$, and incurs polynomial simulation overhead. A timeout wrapper counts down in binary: before the one-tape simulation it uses $O(T \log T)$ time and $O(|M|)$ description length. These are conventions, not exact equalities between particular machine encodings.

Definition 1.2 (Decider)**.** A decider is a five-input machine D which halts with one bit on every (n, x, y, a, b), where n is a positive integer in binary and $x, y, a, b \in \{0, 1\}^*$. The first input is the *index*, and

$$\mathrm{TIME}_D(n) = \max_{x,y,a,b} \mathrm{TIME}_{D,(n,x,y,a,b)}\,.$$

The maximum is understood in $\mathbb{N} \cup \{\infty\}$; a finite bound means that the machine cannot spend time reading arbitrarily remote symbols of the four unrestricted input tapes. Output 1 is acceptance and 0 is rejection.

This is exactly `def:decider`. [2]

[1] Ground truth: `sec:tms` in `ground-truth/gt-03-prelim.tex`.
[2] Ground truth: `def:decider` in `ground-truth/gt-05-games-normalform.tex`.

Definition 1.3 (Conditionally linear sampler)**.** A sampler S is a six-input Turing machine. On index n its legal modes are

$$(n, \mathsf{dimension}), \qquad (n, w, \mathsf{marginal}, j, z),$$
$$(n, w, \mathsf{linear}, j, u, y), \quad (n, w, \mathsf{factor}, j, u),$$

where $w \in \{\mathrm{A}, \mathrm{B}\}$. Its outputs must describe the ambient dimension, the j-th marginal, the conditional linear map, and the coordinate indicator of its factor space for a pair of level-ℓ CL functions over $\mathbb{F}_{q(n)}^{s(n)}$. The paper writes $\mathrm{TIME}_S(n)$ for the number of steps before S halts on index n, i.e. the worst legal call at that index. Inputs and outputs which denote integers, finite-field elements, vectors, and modes are represented by fixed binary conventions.

This transcribes `def:sampler`; its four interfaces and the fact that no call uses more than six tapes are explicit in the source. [3]

Definition 1.4 (Normal form and λ-boundedness)**.** A normal-form verifier is $V = (S, D)$, where S is a CL sampler with field size $q(n) = 2$ and D is a decider. Its description length and level are

$$|V| = \max\{|S|, |D|\}, \qquad \mathrm{level}(V) = \mathrm{level}(S).$$

For an integer $\lambda \geq 1$, V is λ-bounded exactly when

$$|V| \leq \lambda, qquad \mathrm{TIME}_S(n) \leq n^{\lambda}, qquad \mathrm{TIME}_D(n) \leq n^{\lambda} \quad (n \geq 2).$$

No bound is required at $n = 1$.

These are `def:normal-ver` and `def:lambda`. [4] In the game V_n, question and answer alphabets are padded to lengths $\mathrm{TIME}_S(n)$ and $\mathrm{TIME}_D(n)$, respectively. The maximum-over-all-inputs definition above is therefore stronger than a promise only about strings of those lengths.

1.2 What a verifier description means

A "description of a verifier" is the ordered pair $(\overline{S}, \overline{D})$ of canonical machine strings. It is neither an oracle for the functions computed by S, D nor an equivalence class of programs. An algorithm receiving it may count its bits, copy it, reject it as malformed, put it on the input tape of a universal simulator, and manufacture a new string with parts of it hardwired. Two extensionally equal deciders can have different $|D|$, running times, and behavior under those source transformations.

The compression theorem makes this interface literal. There is a polynomial-time machine Compress whose input is $(\overline{V}, \lambda)$, with $\overline{V} = (\overline{S}, \overline{D})$, and whose output is the description of a nine-level verifier. It calls, in order,

$$\overline{V^{(1)}} = \mathsf{ComputeIntroVerifier}(\overline{V}, \lambda, 9),$$
$$\overline{V^{(2)}} = \mathsf{ComputeAnsVerifier}(\overline{V^{(1)}}, \lambda, \mu, \gamma),$$
$$\overline{V^{(3)}} = \mathsf{ComputeParrepVerifier}(\overline{V^{(2)}}, \lambda, \tau).$$

Each call returns another pair of machine descriptions. [5] Likewise, `thm:ar` states that ComputeAnsVerifier "outputs the description of the answer reduced verifier," and its proof separately builds the sampler and decider descriptions. [6]

[3] Ground truth: `def:sampler` in `ground-truth/gt-04-cl.tex`.
[4] Ground truth: `def:normal-ver,def:lambda` in `ground-truth/gt-05-games-normalform.tex`.
[5] Ground truth: `fig:compress,thm:compression` in `ground-truth/gt-12-compression.tex`.
[6] Ground truth: `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`.

1.3 Universal and source-level obligations

The following list is the bookkeeping that a homoiconic description language must discharge. Calling the syntax "lambda calculus" removes none of these obligations.

1. **Simulation and timeouts.** Every high-level instruction which calls S, D, or a generated verifier uses a universal machine, with the polynomial overhead and timeout counter from `sec:tms`. Hardwiring must produce finite code with the asserted length.

2. **Introspection.** The introspection decider contains the description of the original sampler and invokes that sampler to obtain its dimension, marginals, linear maps, and factor spaces. The source constructs a constant-size raw universal decider and then hardwires $(\overline{V}, \lambda, \ell)$; it first scans $|V|$ and replaces an overlong description by a trivial verifier. Thus introspection consumes the sampler description, not merely its induced distribution. [7]

3. **Answer reduction.** In Step 1 of `fig:pcpverifier`, the answer-reduced decider computes

$$\mathsf{Circuit} = \mathsf{PaddedSuccinctDecider}(\overline{D}, n, T, Q, \sigma, x, y)$$

   from the *description* of D. The Cook–Levin circuit depends on the transition table even though it is much smaller than the computation tableau. The construction uses $|D| \leq \sigma$ quantitatively. [8]

4. **Compiler composition.** The three machines displayed in the preceding subsection traverse and assemble source descriptions. The proof that Compress is polynomial time uses their running times. The independence of the compressed sampler is also a statement about which source strings are embedded in its output.

5. **Self-reference.** In the Halting section, the eight-input machine $\mathcal{F}$ is run as

$$\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, \lambda, n, x, y, a, b).$$

   It constructs the five-input program obtained by hardwiring its first three arguments, pairs that decider description with a computed sampler description, and passes the pair to Compress. This is a source-level diagonalization, not recursive invocation alone. [9]

Sections 2–4 construct one description language which meets precisely these obligations. The point is not Church–Turing equivalence; the point is a costed equivalence between finite, inspectable descriptions.

2 A lambda calculus with resources

2.1 Syntax, phases, and canonical bytes

Fix a finite set of ground sorts including Bit, Nat, Bytes, Tuple, MachineDesc, and Quoted(A). Function sorts are finite products followed by a result sort. The raw terms of the description

[7] Ground truth: `lem:intro-decider-complexity,thm:introspection` in `ground-truth/gt-08-introspection.tex`.

[8] Ground truth: `fig:pcpverifier,prop:standard-succinct-sat` in `ground-truth/gt-10-answer-reduction.tex`.

[9] Ground truth: `fig:halt_f` in `ground-truth/gt-12-compression.tex`.

language $\mathcal{L}$ are

$$\begin{aligned} t ::= \ & \mathrm{BoundVar}(d, j) \mid \mathrm{Hole}(h, A) \mid \mathrm{Lambda}(r, t) \mid \mathrm{Apply}(t, t^*) \mid \mathrm{Fix}(t) \\ & \mid \mathrm{If}(t, t, t) \mid \mathrm{Prim}(p, B, t^*) \mid \mathrm{Quote}(\mathrm{Closed}(t)) \\ & \mid \mathrm{Eval}(t_{\mathrm{code}}, t^*, F) \mid \mathrm{Specialize}(t_{\mathrm{code}}, \rho). \end{aligned} \tag{2.1}$$

Here t^* is a finite ordered list. $\mathrm{BoundVar}(d, j)$ addresses slot j of the environment frame d binders outward; hence alpha-conversion is absent from the representation. A hole is a named, typed, static substitution site, not a dynamic variable. We impose the *affine-hole discipline*: each hole name occurs at most once. Reuse of static data is expressed by binding it once and using de Bruijn variables. F is either $\mathrm{FuelLiteral}(m)$ or a closed arithmetic term $\mathrm{FuelBound}(t_n, t_\lambda)$.

Each primitive name p has a total type contract, a deterministic reference machine, and a natural-valued charge $\kappa_p(v_1, \ldots, v_r)$. The serialized annotation B proves $\kappa_p(v) \le B(|v_1|, \ldots, |v_r|)$. The registry contains the usual finite byte, bit, tuple, list, comparison, and arithmetic operations, as well as the bounded machine-table operation used in Theorem 3.1. A primitive is not an oracle: its reference machine runs in at most $c_p(1 + \kappa_p + \sum_i |v_i|)^{e_p}$ Turing steps, with c_p, e_p fixed in the language definition.

The sorting judgment $\Gamma; H \vdash t : A$ is the ordinary simply typed call-by-value judgment for variables, abstraction, application, and conditionals, extended as follows. H records affine holes; Quote requires a scoped closed term with empty H; Specialize requires an environment covering exactly H with terms of the declared sorts; Eval requires quoted code, arguments matching the quoted function sort, and natural fuel. $\mathrm{Fix}(P)$ requires a partial body $P : A$ with the single hole $\mathsf{self\_code} : \mathsf{Quoted}(A)$, and closes that hole. We write $P\{A\}$ for sorted syntax, $\mathrm{Closed}(P)$ for empty variable and hole contexts, and $\mathrm{Quoted}\{A\}$ for its canonical bytes. Raw ill-sorted syntax is retained so that the evaluator can return $\mathsf{TypeError}$ rather than throw a host exception.

Definition 2.1 (Canonical serialization)**.** Let $\nu(m) = 1^\ell 0 b$, where b is the ℓ-bit binary expansion of $m+1$ and $\ell = \lfloor \log_2(m+1) \rfloor + 1$. Thus $|\nu(m)| = 2\ell + 1$, and ν is prefix-free. Encode a byte string z as $\nu(|z|)z$. Encode a term by a four-bit constructor tag followed, in the order displayed in (2.1), by ν-encoded integer and string fields, the encoded child count, and the recursive child encodings. Sorts, primitive contracts, fuel expressions, and static environments use the same rule with fixed tags and lexicographically ordered field names. No node contains a redundant subtree length.

The resulting map ser is injective and self-delimiting. For a term or code value t, define $|t| := |\,\mathrm{ser}(t)|$ in bits. This is the description length used below.

Prefix-freeness of ν determines the end of every field; arities determine the number of recursive parses. Induction on the first constructor tag therefore gives a unique parse. In particular, description size is a number computed from syntax, not the size of a runtime closure.

The operative representations are:

Object	Contents	Legitimate consumer
Closure	Body pointer and runtime environment	Evaluator only; it has no canonical project byte string
Quoted syntax	Canonical bytes of closed $\mathcal{L}$ syntax	Evaluator, specializer, compiler, and Compress
Circuit	Finite Boolean DAG with named input blocks	Succinct clause evaluator and Tseitin transformation
Universal evaluator	One fixed program interpreting quoted syntax	Timeout and bounded-trace construction
Specialization	Capture-free code-to-code replacement	Source transformations; its result is syntax, not a closure

2.2 Small-step semantics and fuel

We use a deterministic call-by-value CEK machine. A runtime closure is $[\lambda r.t, \eta]$, where $\eta = (\eta_0, \eta_1, \ldots)$ is a stack of frames and $\eta_d[j]$ supplies $\mathrm{BoundVar}(d, j)$. Other values are canonical ground data and $\mathrm{Code}(t)$ for closed syntax. A configuration is

$$C = \langle q, \eta, K, H, f \rangle,$$

where q is the current term or value, K is a continuation stack, H is an append-only heap for lists, tuples, closures, and syntax pointers, and $f \in \mathbb{N}$ is remaining fuel. Arguments are evaluated left to right.

For reference, the contractions after all indicated subterms have become values are the following. The notation $C \longrightarrow_k C'$ means that the transition has charge k; continuations and heaps not shown are carried unchanged.

$$\begin{array}{rcll}
\langle \mathrm{BoundVar}(d,j), \eta, K, H, f\rangle & \longrightarrow_1 & \langle \eta_d[j], \eta, K, H, f-1\rangle & \text{if the address exists,} \\
\langle \mathrm{Lambda}(r,t), \eta, K, H, f\rangle & \longrightarrow_1 & \langle [\lambda r.t, \eta], \eta, K, H, f-1\rangle, & \\
\langle \mathrm{Apply}([\lambda r.t, \eta], \vec{v}), K, H, f\rangle & \longrightarrow_1 & \langle t, (\vec{v}) :: \eta, K, H, f-1\rangle & (|\vec{v}| = r), \\
\langle \mathrm{If}(1, v_1, v_0), K, H, f\rangle & \longrightarrow_1 & \langle v_1, K, H, f-1\rangle, & \\
\langle \mathrm{If}(0, v_1, v_0), K, H, f\rangle & \longrightarrow_1 & \langle v_0, K, H, f-1\rangle, & \\
\langle \mathrm{Prim}(p, B, \vec{v}), K, H, f\rangle & \longrightarrow_{\kappa_p(\vec{v})} & \langle p(\vec{v}), K, H, f-\kappa_p(\vec{v})\rangle, & \\
\langle \mathrm{Quote}(\mathrm{Closed}(t)), K, H, f\rangle & \longrightarrow_1 & \langle \mathrm{Code}(t), K, H, f-1\rangle. &
\end{array} \tag{2.2}$$

An absent variable address, failed primitive contract, nonbit condition, wrong application arity, or noncode argument to Eval/Specialize has the sole successor TypeError at charge one. Evaluation contexts supply the omitted rules: for example, Apply first pushes a frame remembering its unevaluated arguments, evaluates the operator, then evaluates each argument from left to right. Thus the display specifies contractions while the context discipline specifies their unique order.

All administrative CEK transitions cost one unit. They include variable lookup, creating a closure, pushing or popping one continuation frame, selecting a Boolean branch, and returning a value. Beta reduction of $\mathrm{Apply}([\lambda r.t, \eta], v_1, \ldots, v_r)$ costs one and continues with t in environment $((v_1, \ldots, v_r), \eta_0, \eta_1, \ldots)$. A wrong arity, applying a nonclosure, or using a nonbit condition takes one transition to TypeError. A residual Hole does the same. $\mathrm{Quote}(\mathrm{Closed}(t))$ takes one transition to the syntax pointer $\mathrm{Code}(t)$; serialization was performed when the enclosing description was admitted, so this transition does not copy $|t|$ bits.

After its arguments have become values, $\mathrm{Prim}(p, B, v_1, \ldots, v_r)$ costs exactly $\kappa_p(v_1, \ldots, v_r) \geq 1$. If the contract or dynamic sorts fail it uses one unit and returns TypeError. For purposes of the local semantics, a charge k is expanded into k deterministic microsteps of the primitive reference machine; thus every microstep consumes one fuel unit.

Specialize($\mathrm{Code}(P), \rho$) first checks the sort and coverage of ρ, then traverses P in prefix order. It uses $1 + |P| + \sum_h |\rho(h)|$ fuel and returns the code value of the substituted term, or TypeError. Eval evaluates its code, arguments, and fuel expression, installs a delimiter holding the requested inner budget, and runs the same CEK transition relation inside it. Installing and removing that delimiter costs two units. Each inner microstep decrements both the inner budget and the ambient budget. Exhausting either produces OutOfFuel.

Finally, $\mathrm{Fix}(P)$ is a value of the result sort of P. When it is applied to arguments u, one unfolding transition installs the static binding

$$\mathsf{self\_code} \longmapsto \mathrm{Code}(\mathrm{Fix}(P))$$

and evaluates P with the same arguments. This is a constant-size heap link, not a serialization of a recursively expanded tree. Including the application and binding frames, an unfolding costs the fixed constant $c_Y = 3$. The corresponding fully specialized syntax can be materialized by Specialize when a source consumer asks for it.

If a transition has charge k and $f < k$, its unique successor is OutOfFuel; otherwise the successor has counter $f - k$. A final configuration is exactly one of

$$\mathsf{Value}(v), \qquad \mathsf{OutOfFuel}, \qquad \mathsf{TypeError}.$$

We write $\mathrm{run}(t; f)$ for this result. For closed function syntax t and an argument tuple u, $\mathrm{eval}_{\mathcal{L}}(t, u; f)$ abbreviates the run of $\mathrm{Apply}(t, \overline{u})$, where $\overline{u}$ is the literal value syntax. Results do not record unused fuel.

Theorem 2.2 (Determinism and outcome trichotomy)**.** *Every nonfinal configuration with positive fuel has at most one successor. For every closed raw term and finite initial fuel, evaluation terminates in exactly one of* Value(v)*,* OutOfFuel*, or* TypeError*.*

Proof. The current control item is either a term, a value, or a primitive microstate. Constructor tags are disjoint. For a term tag, the evaluation-context rule selects the leftmost child not yet represented by a value; if there is one, exactly one continuation frame is pushed. If all children are values, exactly one contraction above applies. Variable addresses select at most one frame and slot. Primitive reference machines and their contracts are deterministic by definition. The Boolean and arity tests have disjoint success and error cases. The same reasoning applies to the unique top Eval delimiter and to the single self-code binding of Fix. Hence two distinct successors cannot exist.

Every nonfinal transition consumes at least one unit. Starting from fuel f, at most f transitions or primitive microsteps can occur before a value or error is returned; if neither has occurred, the next requested step returns out-of-fuel. The three final tags are disjoint, and determinism makes the reached tag unique. □

Theorem 2.3 (Fuel monotonicity)**.** *If* $\mathrm{run}(t; f) = R$ *with* $R \neq$ OutOfFuel*, then* $\mathrm{run}(t; f + g) = R$ *for every* $g \geq 0$*. More precisely, if the first run uses* $c \leq f$ *units, the second uses the same* c *transitions and leaves* $g + (f - c)$ *units.*

Proof. Induct on the c transitions of the terminating run. At step i, the larger-fuel run has exactly g more ambient fuel and, inside each Eval delimiter, exactly the same inner budget as the smaller run. The smaller run can pay the uniquely determined charge k_i, so the larger run can pay it as well. Determinism gives the same nonfuel successor data, and subtraction preserves the difference g. After c steps the larger run therefore reaches the same final tag and value. No operational rule branches on the numeric fuel except the insufficient-fuel rule, which was not selected in the smaller run. □

2.3 Quotation, evaluation, and specialization

External evaluation of bytes must pay for decoding even though an internal Quote is a syntax pointer. Define

$$h(d, u) = 3 + |d| + |\operatorname{enc}(u)|.$$

The quoted evaluator $\mathrm{Eval}_{\mathcal{L}}(d, u; F)$ checks and decodes d, installs the argument tuple, and then runs the CEK machine. By definition the front-end scan uses exactly $h(d, u)$ fuel; malformed code returns TypeError during that scan.

Theorem 2.4 (Quote/Eval equation with explicit overhead)**.** *Let t be a closed function term, let $d = \operatorname{ser}(t)$, and let u have the required argument sorts. For every $f \geq 0$,*

$$\mathrm{Eval}_{\mathcal{L}}(\mathrm{Quote}(t), u; f + h(d, u)) = \mathrm{eval}_{\mathcal{L}}(t, u; f). \tag{2.3}$$

Here the left side takes the canonical bytes denoted by Quote(t)*. If the right side uses $c \leq f$ units and returns a value or type error, the left side uses exactly $h(d, u) + c$. If it requests a $(f + 1)$-st unit, both sides return out-of-fuel after their respective budgets are exhausted.*

Proof. The prefix decoder is inverse to serialization by Definition 2.1; hence after its fixed scan the left machine holds the same syntax tree t, literal arguments, empty lexical environment, and application continuation as the right machine. The left counter is then exactly f. From this point the two configurations are identical. Theorem 2.2 gives identical successors until a final outcome. The front-end accounts for the stated additional $h(d, u)$ units and no other rule differs. □

Lemma 2.5 (Capture-free specialization and its size)**.** *Let P be well-scoped partial syntax satisfying the affine-hole discipline, and let σ map every hole of P to closed, sort-correct syntax. There is a unique term* Specialize(P, σ)*, it has no holes, it preserves the sort of P, and*

$$|\operatorname{Specialize}(P, \sigma)| \leq |P| + \sum_{h \in \operatorname{dom} \sigma} |\sigma(h)|. \tag{2.4}$$

The construction takes at most $1 + |P| + \sum_h |\sigma(h)|$ CEK microsteps.

Proof. Traverse P in prefix order. Copy every nonhole tag and payload unchanged. At $\mathrm{Hole}(h, A)$, check the unique table entry, check its sort, and emit $\sigma(h)$. The inserted term is closed, so its de Bruijn indices refer to no binder in P; indices in the surrounding syntax are also unchanged. Thus no variable can be captured. Induction on P proves sort preservation, because the hole and replacement have the same sort at the only new case. Coverage removes every hole, and deterministic traversal gives uniqueness.

Serialization has no ancestor subtree-length fields. Replacing the one occurrence of h deletes a nonempty encoded Hole node and inserts exactly $|\sigma(h)|$ bits. Since distinct affine holes have disjoint occurrences,

$$|\operatorname{Specialize}(P,\sigma)| = |P| - \sum_h |\operatorname{Hole}(h, A_h)| + \sum_h |\sigma(h)|,$$

which implies (2.4). One scan step per input or emitted bit, plus the initial check, gives the time bound. Without the affine-hole discipline the correct last term would sum once per hole occurrence; this is why the discipline is part of the formal system. □

3 Simulation theorems: the machine–term bridge

3.1 From the paper's machines to $\mathcal{L}$

We use explicit functional tapes and one bounded table primitive. A tape is a zipper (L, s, R): L is the reversed list strictly left of the head, $s \in \{0, 1, \sqcup\}$ is the scanned symbol, and R is the list strictly right of it with implicit blank tail. A right move maps (L, s, rR') to (sL, r, R'), taking $r = \sqcup$ when R is empty; a left move is symmetric. These are fixed list primitives of charge at most four. A configuration is a tuple of the finite control state and the $k + 2$ zippers for input, work, and output tapes.

For a machine description m, the primitive $\mathsf{TMDelta}(m, q, s_1, \ldots, s_{k+2})$ scans the encoded transition table and returns the next state, written symbols, and head motions. Its declared charge is

$$\kappa_{\mathsf{TMDelta}} \le 8(|m| + k + 2).$$

The primitive validates m; malformed descriptions use a fixed rejecting transition. Its implementation is an ordinary finite table scan. Thus the description of the table remains data of length $|m|$, rather than being duplicated into a nest of conditionals.

Let init_k parse the k-tuple encoding into input zippers and blank work/output zippers, and let out read the longest nonblank output prefix. Define

$$\begin{aligned} \mathsf{loop}_m &= \operatorname{Fix}\big(\lambda C.\ \operatorname{If}(\mathsf{halt}(C), \mathsf{out}(C), \\ &\qquad\qquad \mathsf{self}(\mathsf{move}(C, \mathsf{TMDelta}(m, \mathsf{view}(C)))))\big), \\ \langle\!\langle M \rangle\!\rangle &= \lambda z.\mathsf{loop}_{\overline{M}}(\mathsf{init}_k(z)). \end{aligned} \tag{3.1}$$

Here self is the closure obtained from the Fix self-code link; equivalently it is Eval of that link with the next configuration. All other displayed operations are fixed primitives with the stated list representation.

Theorem 3.1 (TM to $\mathcal{L}$, with resources)**.** *There are universal constants $a_0, b_0, c_0 \ge 1$ such that for every k-input machine M,* (3.1) *is a closed term and*

$$|\langle\!\langle M \rangle\!\rangle| \le a_0|M| + b_0.$$

If $M(x_1, \ldots, x_k)$ halts within $T \ge 1$ transitions, then

$$\operatorname{Eval}_{\mathcal{L}}(\operatorname{Quote}(\langle\!\langle M \rangle\!\rangle), \langle x_1, \ldots, x_k \rangle; c_M T(T + |x|)) = M(x_1, \ldots, x_k), \tag{3.2}$$

where $|x| = \sum_i |x_i|$ *and one may take* $c_M = c_0(|M| + k + 1)$*. The tuple on the left is the same dual-rail encoding as in Definition 1.1. In particular, a decider receives exactly*

$$\langle \mathrm{bin}(n), x, y, a, b\rangle_{\mathcal{L}}, \qquad \mathrm{enc}_{\mathcal{L}}(\langle \mathrm{bin}(n), x, y, a, b\rangle_{\mathcal{L}}) = \mathrm{enc}_5(\mathrm{bin}(n), x, y, a, b),$$

and the five components initialize five separate input zippers.

Proof. The term contains one copy of $\overline{M}$, the fixed interpreter body, and self-delimiting constructor overhead. Each input bit in $\overline{M}$ contributes a fixed number of serialized bits, so constants a_0, b_0 independent of M give the size bound.

Initialization scans the dual-rail input once. For valid input it consumes at most $c_0(|x| + k + 1)$ units and produces exactly the paper's initial tapes. Assume inductively that the functional configuration before loop iteration i represents the machine configuration after i transitions. The views return the same scanned symbols. The table primitive selects the unique transition of M, and each zipper update writes the selected symbol and moves in the selected direction. Therefore the next functional configuration represents the machine configuration after $i + 1$ transitions. This proves the simulation invariant for $0 \le i \le T$.

One loop iteration, including the fixed-point unfolding, table scan, tuple operations, and $k + 2$ zipper updates, uses at most $c_0(|M| + k + 1)$ units. No visited zipper has more than $|x| + T$ explicit cells. The final output scan therefore uses at most $c_0(|x| + T + 1)$ units. Adding initialization, T iterations, output, and the quote/decode overhead from Theorem 2.4 gives at most

$$c_0(|M| + k + 1)(|x| + 1 + T) + c_0(|M| + k + 1)T.$$

For $T \ge 1$, both $|x| + T + 1$ and T are at most $2T(T + |x|)$. Increasing the universal choice of c_0 by a factor four makes this no greater than the fuel in (3.2). The invariant and the definition of out then give the same output string. The explicit quadratic form is deliberately conservative; the zipper simulation itself is linear in T apart from the hardwired table scan. □

3.2 From $\mathcal{L}$ to the paper's machines

Theorem 3.2 (A universal evaluator for $\mathcal{L}$)**.** *There is a fixed three-input Turing machine* $U_{\mathcal{L}}$*, of description length* $b_U = O(1)$*, and a universal constant* $C_L \ge 1$*, such that on input* (d, u, f) *it returns the same value, out-of-fuel, or type-error result as* $\mathrm{Eval}_{\mathcal{L}}(d, u; f)$*, and*

$$\mathrm{TIME}_{U_{\mathcal{L}}}(d, u, f) \le C_L\big(|d| + |u| + f + 1\big)^6. \tag{3.3}$$

Given closed t*, hardwiring* $d = \mathrm{ser}(t)$ *produces a machine* $(U_{\mathcal{L}})_t$ *with*

$$|(U_{\mathcal{L}})_t| \le a_U|t| + b_U.$$

On the paper's separate input tapes it may represent each argument by a lazy head-and-position handle. Consequently, a run with fuel f *accesses at most* f *symbols of those tapes and has the refined bound* $C_L(|t| + f + 1)^6$*, independent of unread suffixes. Hence a total decider term with fuel schedule* $f_D(n)$ *is a decider in the paper's sense and*

$$\mathrm{TIME}_D(n) \le C_L(|t| + f_D(n) + 1)^6. \tag{3.4}$$

Proof. The machine first checks the prefix serialization and writes node records on a work-tape heap. It stores a node address, environment-frame addresses, continuation records, and binary fuel counters. During at most f microsteps the heap acquires at most a fixed number of records per step, so its length is at most $c(|d| + |u| + f + 1)$ for a universal c. A conservative single-tape implementation finds a record by a full scan, performs binary address arithmetic, and returns to the simulated head in at most the square of that length. Decoding, f simulated steps, and final serialization are therefore bounded by a universal constant times the fourth power of the length. Converting the fixed multitape implementation to the paper's one-tape convention and allowing the paper's polynomial universal-simulation overhead is bounded by the sixth power after increasing C_L. This proves (3.3). Primitive microstates use their fixed reference machines and have charged running time; their polynomial exponents were fixed when $\mathcal{L}$ was fixed and are included in the exponent six (or, for a different registry, in another fixed exponent).

The simulator description contains only the parser, CEK transition table, primitive registry, and serializers, so its length is b_U, independent of d, u, f. The paper's hardwiring convention embeds d once and gives the linear description bound. For separate input tapes, a handle stores a tape number and head position. One microstep can move such a head by at most the work performed during that charged step, so no more than f input symbols can be inspected. Removing the untouched encoded suffixes from the preceding storage account yields the refined bound. Taking a maximum over all (x, y, a, b) now proves (3.4); arbitrarily long unread suffixes do not affect it. □

3.3 The resource dictionary

For a term program, let $s_t = |t|$ and let $f_t(n)$ be a proved fuel schedule which suffices for every legal call at index n. The translations above give the following dictionary.

Paper quantity	$\mathcal{L}$ counterpart	Quantitative relation
$\mathrm{TIME}_D(n)$	decider fuel $f_D(n)$	Term to TM: $\mathrm{TIME}_D(n) \le C_L(\|D_{\mathcal{L}}\| + f_D(n) + 1)^6$. TM to term: fuel $c_D T(T + \|u\|)$.
$\mathrm{TIME}_S(n)$	maximum sampler-mode fuel $f_S(n)$	Same two bounds, taking the maximum over the dimension, marginal, linear, and factor calls.
$\|D\|$	$\|\mathrm{ser}(D_{\mathcal{L}})\|$	Each translation is linear: at most $a_0\|D\| + b_0$ or $a_U\|D_{\mathcal{L}}\| + b_U$.
$\|S\|$	$\|\mathrm{ser}(S_{\mathcal{L}})\|$	The same linear bounds; static CL tables and source descriptions are counted once.
λ-bounded	size at most λ, fuel at most n^λ	Preserved in both directions as $A\lambda$-bounded for a universal integer A, as proved below.
Threshold $n \ge C_0$	same semantic index condition	Translation does not change n; resource estimates may replace it by $n \ge \max\{C_0, 2\}$, never hide it in poly.

Theorem 3.3 (Preservation of λ-boundedness)**.** *Fix the encodings and primitive registry above. There is a universal integer $A \ge 1$ such that:*

1. *translating any λ-bounded paper verifier term-by-term with Theorem 3.1 gives an $A\lambda$-bounded $\mathcal{L}$-verifier;*
2. *compiling any λ-bounded $\mathcal{L}$-verifier with Theorem 3.2 gives an $A\lambda$-bounded paper verifier.*

The translation preserves the verifier's index and its input/output function.

Proof. For the first direction, $|M| \le \lambda$ and the size theorem gives $|\langle\langle M \rangle\rangle| \le (a_0 + b_0)\lambda$, since $\lambda \ge 1$. At index $n \ge 2$, the paper machine runs for $T \le n^\lambda$. It can inspect or emit at most T tape symbols. Across at most six input tapes the portion relevant to the computation therefore has encoded length at most $8n^\lambda$, after increasing the constant to cover $\log n$ and mode tags. Also $c_M \le c_0(\lambda + 7) \le 8c_0 n^\lambda$. Theorem 3.1 then requires at most

$$8c_0 n^\lambda \cdot n^\lambda \cdot 9n^\lambda = 72c_0 n^{3\lambda} \le n^{(3+\lceil \log_2(72c_0) \rceil)\lambda}.$$

The last inequality uses $n \ge 2$ and $\lambda \ge 1$. It applies to both the sampler and decider.

For the second direction, hardwiring gives description size at most $(a_U + b_U)\lambda$. With fuel at most n^λ, the lazy-input form of (3.4) is at most

$$C_L(\lambda + n^\lambda + 1)^6 \le C_L(3n^\lambda)^6 \le n^{(6+\lceil \log_2(C_L 3^6) \rceil)\lambda}.$$

We used $\lambda \le 2^\lambda \le n^\lambda$. The same calculation holds for every sampler mode. Taking A to be the maximum of the four displayed description and exponent coefficients proves both assertions. Semantics is preserved by Theorems 3.1 and 3.2; neither translation changes the binary index. □

The constants $a_0, b_0, c_0, a_U, b_U, C_L$, fixed arities, serialization tags, and the universal simulation exponent are absorbed when the paper writes $O(\cdot)$ or $\mathrm{poly}(\cdot)$: they are universal after the two languages are fixed. The particular $|D|, |S|$, a supplied λ, a fuel or time parameter, and hardwired $|M|$ are *arguments* of those polynomials and cannot be hidden as universal constants. Nor can the semantic threshold C_0, the change from λ to $A\lambda$, or constants appearing inside completeness and soundness inequalities be erased by poly notation.

4 Self-reference as a construction on descriptions

4.1 The recursion theorem with size accounting

Let $\varphi_e^{(k)}$ be the partial function of the k-input paper machine with description e. We use equality of partial functions only in the conclusion; every operation in the construction acts on strings.

Lemma 4.1 (The *s-m-n* operation)**.** *For every $r, k \ge 1$ there is a total source transformer $s_{r,k}$ such that*

$$\varphi^{(k)}_{s_{r,k}(e,z_1,\ldots,z_r)}(x) = \varphi_e^{(r+k)}(z_1, \ldots, z_r, x).$$

Under the paper's hardwiring convention there are constants a_s, b_s, depending only on r, k, for which

$$|s_{r,k}(e, z)| \le a_s(|e| + \textstyle\sum_i |z_i|) + b_s,$$

and the output code is computed in time linear in the right-hand side.

Proof. Output the fixed wrapper which writes $\mathrm{enc}_{r+k+1}(e, z_1, \ldots, z_r, x)$ on its work tape and invokes the universal machine U_{r+k}. The wrapper contains one copy of each static bit and constant code for tuple encoding and universal simulation. It therefore has the displayed linear length and can be printed in linear time. The universal-machine theorem gives the stated partial-function equation on halting inputs; on a nonhalting input both sides simulate the same machine forever. □

Theorem 4.2 (Kleene's second recursion theorem, paper model)**.** *Let Q be a total computable transformation from descriptions of k-input machines to descriptions of k-input machines. There is an effectively constructible description e_Q such that*

$$\varphi^{(k)}_{e_Q} = \varphi^{(k)}_{Q(e_Q)}. \tag{4.1}$$

If q is a description of the machine computing Q, then, for constants a_K, b_K fixed by k and the encodings,

$$|e_Q| \le a_K|q| + b_K,$$

and e_Q is constructed in time polynomial in $|q|$.

Proof. By Lemma 4.1, let $s(z, z)$ denote the description obtained by hardwiring two copies of z into the first two inputs of a suitable $(k + 2)$-input machine. Define the partial computable function

$$G(z, x) = \varphi^{(k)}_{Q(s(z,z))}(x).$$

A machine for G first computes the finite string $s(z, z)$, runs the total code transformer Q on it, and universally evaluates the resulting k-input code on x. Let p be its description and set $e_Q = s(p, p)$. Then for every x, including the case in which the final computation diverges,

$$\varphi^{(k)}_{e_Q}(x) = \varphi^{(k+1)}_p(p, x) = \varphi^{(k)}_{Q(s(p,p))}(x) = \varphi^{(k)}_{Q(e_Q)}(x),$$

which proves (4.1).

The program p contains one copy of q and fixed code for s and the universal evaluator, so $|p| \le a_1|q| + b_1$. Applying the linear size bound of Lemma 4.1 to $s(p, p)$ gives $|e_Q| \le 2a_s a_1|q| + (2a_s b_1 + b_s)$. These are the asserted a_K, b_K. Printing the two wrappers and executing the total hardwiring transformations takes polynomial time under the paper's conventions. □

4.2 The call-by-value Z combinator and YCode

For orientation, call-by-value lambda calculus uses

$$Z = \lambda f.(\lambda x.f(\lambda u.xxu))(\lambda x.f(\lambda u.xxu)),$$

rather than the call-by-name Y, because eager evaluation of Yf unfolds before supplying an argument. This untyped term has constant size, but it does not itself provide a canonical code value of the fixed point. Our typed description constructor does.

For a partial body P with its one distinguished quoted hole, define

$$\mathrm{YCode}(P) := \mathrm{Fix}(P), \qquad d_P := \mathrm{Quote}(\mathrm{YCode}(P)).$$

The source equation is

$$\mathrm{unfold}(\mathrm{YCode}(P)) = \mathrm{Specialize}(P, \{\mathsf{self\_code} \mapsto d_P\}). \tag{4.2}$$

It is an equation between finite descriptions. Its right side is produced on demand by Lemma 2.5; the runtime evaluator instead uses a constant-size environment link.

Theorem 4.3 (Description-level fixed-point equation)**.** *For every well-sorted P, argument tuple u, and $f \geq c_Y = 3$,*

$$\mathrm{eval}_{\mathcal{L}}(\mathrm{YCode}(P), u; f) = \mathrm{eval}_{\mathcal{L}}(\mathrm{Specialize}(P, \{\mathsf{self\_code} \mapsto d_P\}), u; f - c_Y)\,. \tag{4.3}$$

Equivalently, if f_P is the code transformer represented by P, the right side is conventionally written $\mathrm{eval}_{\mathcal{L}}(f_P(\mathrm{YCode}(f_P)), u; f - c_Y)$*. Moreover*

$$|\,\mathrm{YCode}(P)| = |P| + c_{\mathrm{fix}}$$

for the fixed four-bit Fix tag and its fixed sort annotation $c_{\mathrm{fix}} = O(1)$.

Proof. The unique Fix application rule performs three administrative actions: push the application frame, install the self-code heap link, and enter P. The resulting control, ordinary environment, and continuation are the same as for the specialized body; a lookup of the sole hole follows the installed link and therefore returns d_P. Subsequent transitions coincide by Theorem 2.2. Exactly three units have been consumed, which proves (4.3), including matching out-of-fuel and type-error outcomes. Canonical serialization of Fix writes its fixed tag and annotation followed by one copy of the serialization of P; it never expands the self-code link. This proves the size equality. □

4.3 The paper's explicit diagonal program

Let $M : P\{\mathsf{MachineDesc}\}$ and $L : P\{\mathsf{Level}\}$ be closed literal terms, let S_L be the sampler description returned by $\mathsf{ComputeSampler}(L)$, and let n, x, y, a, b be the five bound inputs. The partial body corresponding to `fig:halt_f` is

$$\begin{aligned}\Psi_{M,L} = \lambda(n,x,y,a,b).\ \mathrm{If}(\,&\mathrm{Prim}(\mathsf{halts\_within}, M, n),\ \mathsf{true},\\ &\mathrm{Eval}(\mathrm{ans}(\mathrm{Eval}(\mathrm{Quote}(\mathsf{Compress}),\\ &\quad(\mathrm{quoted\_pair}(\mathrm{Quote}(S_L), \mathrm{Hole}(\mathsf{self\_code}, \mathsf{Quoted}(\mathsf{Decider}))), L), F_C)),\\ &\quad(n,x,y,a,b), \mathrm{FuelBound}(n, L))).\end{aligned} \tag{4.4}$$

The finite-fuel symbol F_C is any proved bound for the terminating $\mathsf{Compress}$ call; ans selects the returned decider description. Define

$$D_{M,L} = \mathrm{YCode}(\Psi_{M,L}), \qquad V_{M,L} = (S_L, \mathrm{Quote}(D_{M,L})). \tag{4.5}$$

Theorem 4.4 (Equivalence of the three fixed-point presentations)**.** *The following are translations of the same description-level construction:*

1. *the paper's explicit $\mathcal{F}(\overline{\mathcal{F}}, \overline{M}, L, n, x, y, a, b)$;*
2. *the Kleene fixed point of the transformer which hardwires $(\overline{M}, L)$ and inserts the resulting decider code into* $\mathsf{Compress}$*;*
3. *the closed description $D_{M,L}$ in* (4.5)*.*

Under the simulations of Section 3, their deciders compute the same partial function. Their description lengths are all $O(|M| + \log L + 1)$, and their running times differ by a polynomial in $|M|, \log L, n$, the invoked fuel bounds, and the invoked machine times.

Proof. In the paper, Step 2 of $\mathcal{F}$ applies the hardwiring operation to the eight-input code supplied in its first argument. When that argument is $\overline{\mathcal{F}}$, the resulting five-input code is precisely the code of the outer decider $D(n, x, y, a, b) = \mathcal{F}(\overline{\mathcal{F}}, \overline{M}, L, n, x, y, a, b)$. Thus the string given to Compress is its own decider description. This is the concrete *s-m-n* operation of Lemma 4.1 with the diagonal argument already chosen.

Alternatively, let $Q(e)$ output the code which performs the halting test and, on failure, applies Compress to (S_L, e) and runs the returned decider. Theorem 4.2 gives e_Q with $\varphi_{e_Q} = \varphi_{Q(e_Q)}$, which is exactly the behavior just described. Equation (4.2) performs the same substitution with e_Q represented by d_P; Theorem 4.3 proves the operational equation for $D_{M,L}$. Translating it to a machine with Theorem 3.2, or translating $\mathcal{F}$ with Theorem 3.1, preserves that behavior.

The displayed term contains one copy of M, one binary literal L, and fixed code for the remaining operations. Theorem 4.3 therefore gives $|D_{M,L}| = O(|M| + \log L + 1)$. The paper's hardwiring and the size calculation in Theorem 4.2 give the same bound for the first two forms. Finally, the two simulation theorems replace each bounded call by a fixed polynomial overhead. Composition of finitely many such polynomials is a polynomial in the listed quantities, proving the time statement. □

The paper notes that an earlier version used Kleene's recursion theorem and that its published presentation uses $\mathcal{F}$ explicitly. [10] Theorem 4.4 explains why those presentations differ in notation rather than in the description passed to compression.

What must Compress do with the quoted body? It first applies Specialize to the static verifier and parameter holes, obtaining closed syntax with the size bound of Lemma 2.5. It then traverses that syntax to build the descriptions for introspection, answer reduction, and repetition, and it records the resulting byte lengths and fuel bounds. It never asks whether two terms compute the same function. Such extensional equality is neither available nor needed. This self-reference is not the hard part: it supplies no low-degree soundness, rigidity, repetition, or gap preservation. Those remain the substantive cited theorems.

5 Cook–Levin for $\mathcal{L}$-descriptions

5.1 A local encoding of bounded evaluation

For closed t, input u, and fuel F, the *bounded trace* is

$$\mathcal{T}(t, u, F) = (C_0, C_1, \ldots, C_r), \qquad r \le F,$$

where C_0 is the initial CEK configuration and each C_{i+1} is its unique small-step successor. A row records the current term pointer, environment-frame pointers, continuation, heap, primitive microstate, outcome flag, and binary fuel. If evaluation halts early, a fixed self-looping final state pads the trace to $F + 1$ rows. This is the promised sequence of (term, environment, fuel) configurations, with the continuation and heap made explicit so that one-step validity is local.

To obtain a finite-alphabet tableau, compile the CEK evaluator and each charged primitive to a fixed multitape Turing machine $\widehat{U}_{\mathcal{L}}$. The read-only code track contains $\mathrm{ser}(t)$; input tracks hold lazy argument tapes; work tracks hold the environment, heap, stack, and fuel. Pointer arithmetic,

[10] Ground truth: `fig:halt_f` in `ground-truth/gt-12-compression.tex`.

traversal, and a primitive of charge k are expanded into their elementary head moves. One microstep changes the finite control, the cells under the heads, and the head positions by at most one.

Lemma 5.1 (Transition-window lemma)**.** *There is a finite alphabet $\Gamma_{\mathcal{L}}$, a fixed number r_0 of tracks, and a Boolean function*

$$\tau : (\Gamma_{\mathcal{L}}^3)^{r_0} \times Q_{\mathcal{L}} \longrightarrow \Gamma_{\mathcal{L}}^{r_0} \times Q_{\mathcal{L}}$$

with the following property. For every nonboundary cell j, the symbols in cell j of row $i+1$, together with the next finite-control state, are determined by the radius-one windows $(j-1, j, j+1)$ of row i and the current control state. Boundary cells use a fixed endmarker version of the same rule. Conversely, if two consecutive rows have one head marker on each track, agree with τ at every window, and have valid endmarkers, then the second row is exactly the successor of the first under the small-step semantics.

Proof. Encode a head by pairing the tape symbol with one of the finitely many machine states when that head is the active head, and use a separate finite-control track for the other heads. In one transition, a cell farther than one position from every head is copied. At a head cell, the transition table determines the written symbol and next state from the old state and scanned symbols. At its left and right neighbors, the table determines whether the head marker enters; it cannot enter any other cell because heads move by at most one. These cases depend only on the three displayed cells on each track. They are mutually exclusive once the old row has one head per track, and therefore define the finite function τ. Endmarkers replace missing neighbors by a fixed boundary symbol.

For the converse, consider each track. Away from the unique old head, the local copy case forces equality. In the three-cell neighborhood of that head, the transition case forces the unique symbol write, direction, and new head position specified by the machine table. The local rules also force all other cells not to acquire a head. Repeating this argument for every track and using the shared finite-control component gives precisely one transition of $\widehat{U}_{\mathcal{L}}$. Primitive execution and CEK operations introduce no exceptional case because they were compiled into the same elementary machine steps. The correspondence between those microstates and the weighted semantics was part of the construction in Section 2. □

5.2 The succinct 3SAT circuit

Let the row width be W. In F microsteps no head visits beyond the initial code and input plus F cells, so $W = O(|t| + |u| + F)$. Introduce Boolean variables for the constant-size binary encoding of every tableau cell and control-state bit. The formula asserts: the initial row contains t, u, F; every row and track has the prescribed format and head marker; each adjacent pair obeys every transition window; and the last row is accepting. Each assertion involves a constant window except binary addressing and the initial read-only strings. Fixed Tseitin gadgets turn it into 3CNF with $O(FW)$ variables and clauses.

Theorem 5.2 (Succinct Cook–Levin for $\mathcal{L}$)**.** *There is an algorithm which, on $(t, n, F, Q, \sigma, x, y)$, where $|t| \le \sigma$ and $|x|, |y| \le Q$, outputs a Boolean circuit $C_{t,n,F,Q,\sigma,x,y}$ deciding membership of an indexed signed clause in a 3CNF formula Φ. Its index width is $O(\log F + \log \sigma)$, its size and construction time are*

$$\mathrm{poly}(\log n, \log F, Q, \sigma), \tag{5.1}$$

and for every pair of answer strings a, b the formula is satisfiable with the prescribed input bits if and only if $\mathrm{Eval}_{\mathcal{L}}(\mathrm{Quote}(t), (n, x, y, a, b); F)$ accepts. Holding the public input fixed, the transi-

tion component of the circuit has size $\mathrm{poly}(\log F, |t|)$*; the* Q *term in* (5.1) *is solely the hardwired initialization data* x, y*.*

Proof. Given a clause index, the circuit first decodes whether the requested clause is an initialization, uniqueness/format, transition, or acceptance clause. Row and column numbers use $O(\log F + \log W)$ bits. For a transition clause it computes the addresses of a radius-one window and evaluates the finite truth table τ from Lemma 5.1; binary addition, comparison, and multiplexing have size polynomial in those address lengths. For an initialization clause it selects a bit of the canonical code or public input. A balanced selector for a hardwired string has size linear in $|t| + Q + \log n$. Format and final-state clauses use fixed truth tables. Finally it emits the three variable addresses and signs of the selected 3CNF gadget. This proves the circuit and construction bounds. Since $W = O(|t| + Q + F)$, $\log W = O(\log F + \log \sigma + \log(Q + 1))$, which is absorbed by (5.1) under $Q \le F$, as in the paper.

If evaluation accepts, assign every tableau variable from its unique bounded trace and every Tseitin variable from its gadget value. Initialization, transition, and acceptance clauses are then satisfied. Conversely, any satisfying assignment contains the prescribed initial row. Induction on the row number and the converse part of Lemma 5.1 forces each later row to be the unique evaluator successor. The final clauses force that trace to accept. The prescribed answer bits occupy the initial answer tapes, so the equivalence holds for each a, b. □

This is the $\mathcal{L}$-description restatement of `prop:standard-succinct-sat`. The paper's version takes D, n, T, Q, σ, x, y, reserves the first $2T$ encoded positions for each answer, and has size $\mathrm{poly}(\log n, \log T, Q, \sigma)$. [11] Theorem 3.2 gives the same proposition by compilation; the proof above shows directly why a lambda evaluation trace has the required locality.

More explicitly, use the paper's tape-alphabet encoding $\mathrm{enc}_\Gamma(0) = 00$, $\mathrm{enc}_\Gamma(1) = 01$, $\mathrm{enc}_\Gamma(\sqcup) = 10$, and reserve $2F$ bits in the tableau assignment for each answer tape. For every $a, b \in \{0,1\}^{2F}$, the resulting formula has an extension c if and only if there are prefixes a', b', of lengths at most F, such that

$$a = \mathrm{enc}_\Gamma(a', \sqcup^{F-|a'|}), \qquad b = \mathrm{enc}_\Gamma(b', \sqcup^{F-|b'|}),$$

and the term accepts (n, x, y, a', b') within fuel F. The forward direction follows because the initialization clauses force the two reserved blocks to be valid padded tape rows; the backward direction fills those blocks from the accepting trace. This is the exact placement property needed when the two blocks are separated in (5.2).

5.3 From shared 3SAT data to decoupled 5SAT

The paper removes two undesirable features before low-degree encoding. A 3SAT clause reads three locations of one tableau assignment, while three low-degree answers need not be restrictions of one assignment. Also, the strings a, b are embedded inside that assignment, whereas the later sampler must query them as independent blocks. Make five assignments a, b, u_3, u_4, u_5 and impose clauses of the form

$$a_{i_1}^{o_1} \vee b_{i_2}^{o_2} \vee (u_3)_{i_3}^{o_3} \vee (u_4)_{i_4}^{o_4} \vee (u_5)_{i_5}^{o_5}. \tag{5.2}$$

The original 3SAT clauses are applied to u_3; constant-size five-literal gadgets enforce $u_3 = u_4 = u_5$, copy the first $2F$ encoded answer bits to a, b, and pad unused positions by the alternating blank

[11] Ground truth: `prop:standard-succinct-sat` in `ground-truth/gt-10-answer-reduction.tex`.

encoding. Each gadget is indexed by comparisons and additions on $O(\log F + \log \sigma)$-bit addresses, so it adds only that many gates. Therefore the succinct decoupled circuit retains the bound (5.1). Its padded form makes all five blocks have one common power-of-two length. A satisfying five-tuple restricts to a satisfying trace assignment, and a satisfying trace extends by copying and padding; this proves the required if-and-only-if. [12]

The intended analytic chain is

$$\begin{aligned}
\texttt{Quoted\{Decider\}} &\xrightarrow{\texttt{bounded_trace}} \texttt{BoundedTrace} \\
\texttt{BoundedTrace} &\xrightarrow{\texttt{cook_levin}} \texttt{Succinct3SAT} \\
\texttt{Succinct3SAT} &\xrightarrow{\texttt{decouple5}} \texttt{SuccinctDecoupled5SAT} \\
\texttt{SuccinctDecoupled5SAT} &\xrightarrow{\text{padding}} \texttt{PaddedSuccinctDecoupled5SAT}.
\end{aligned}$$

These are the transformations planned for TB3 in `briefs/23-tb3.md`. That brief is planned work, not evidence that the functions exist or that any claim has been checked. At the time of this note, no `bounded_trace`, `cook_levin`, or `decouple5` implementation is present in `src/`; the general contracts remain CITED.

5.4 Tseitin and arithmetization

Given a Boolean circuit C with gates g_i, Tseitin introduces a wire variable w_i and an equivalence formula

$$z_i = (g_i(x, w) \wedge w_i) \vee (\neg g_i(x, w) \wedge \neg w_i), \qquad F = \bigwedge_i z_i.$$

The required contract is $C(x) = 1$ iff there exists w with $F(x, w) = 1$, with size and construction time $O(|C|)$.[13] The project follows the explicit gate-equivalence form in `ground-truth/nw19/nw19-tseitin-arith.tex` and also conjoins the output literal w_{out}, so the stored formula explicitly asserts acceptance. That convention is marked SOURCE_REPAIR; it is not silently attributed to the source.

The Julia constructors `Circuit`, `Input`, `Gate`, `NotGate`, `AndGate`, and `OrGate` make the finite DAG and its topological order explicit. The function `tseitin` returns `Checked{TseitinFormula, CertNode}`. Its replay recomputes the formula's occurrence vector; it does not, by that replay alone, establish the general Cook–Levin or Tseitin semantic equivalence. Those implications are exhausted only on the small TB0 fixture and remain cited in general.

Arithmetization recursively applies

$$\neg A \mapsto 1 - A, \qquad A \wedge B \mapsto AB, \qquad A \vee B \mapsto A + B - AB.$$

It agrees with the Boolean formula on the Boolean cube. [14] The related NW19 rule is `def:arithmetization` in `ground-truth/nw19/nw19-tseitin-arith.tex`. Julia's `arith_q` returns a sparse formal `Poly`; its `ArithTseitin` certificate checks support degrees against a structural bound.

[12] Ground truth: `sec:succinct-deciders` in `ground-truth/gt-10-answer-reduction.tex`.
[13] Ground truth: `def:tseitin` in `ground-truth/gt-10-answer-reduction.tex`.
[14] Ground truth: `def:formula-arithmetization` in `ground-truth/gt-10-answer-reduction.tex`.

5.5 The occurrence-vector discrepancy

There is a discrepancy in the individual-degree bookkeeping as we read NW19; our reading may be wrong. The source proposition states individual degree at most two for the Tseitin arithmetization. [15] Reading the NW19 formula literally as a formula tree, arithmetization is multilinear in leaf occurrences, not necessarily in circuit wires. The same wire can occur in several gate gadgets. With the project's explicit output literal, the count we compute is

$$\mathrm{occ}(x_j) = 2\,\mathrm{fanout}(x_j), \qquad \mathrm{occ}(w_i) = 2 + 2\,\mathrm{fanout}(w_i) + \mathbf{1}_{i=\mathrm{out}},$$

and structural induction gives

$$\deg_v(F_{\mathrm{arith}}) \le \mathrm{occ}_v(F).$$

The function `tseitin_occurrence_account` computes this vector; `occurrences` independently traverses the formula; and `arith_q` compares the structural account with actual sparse support.

Claim C8 is only `C8`: TESTED: the two-gate regression gives $(2, 2, 2, 4, 3)$, and the 16-variable TB0 formula attains

$$(2, 0, 0, 0, 2, 2, 0, 0, 0, 0, 6, 4, 4, 4, 4, 3).$$

The general occurrence formula is a written derivation, not a machine theorem. The downstream soundness repair is only `C5`: SKETCH. With a symbolic bound $\delta_F = \max_v \mathrm{occ}_v(F)$, the conservative formula-test term becomes $(\delta_F + 5d)m'/q$, and `P_formula_structural` is an additional obligation. Under an explicit fanout-reduction hypothesis, $\mathrm{fanout}_{\max} \le 2$ gives $\delta_F \le 7$ with the output literal, and $d \ge 8$ suffices for the individual-degree construction bound. Absorption into the asymptotic parameter choice still needs the stated growth hypotheses; it is not promoted by the two finite computations.

6 The low-degree PCP as algebra

6.1 Interpolation and the vanishing polynomial

For $M = 2^m$ and $a = (a_y)_{y \in \{0,1\}^m}$, define

$$\mathrm{ind}_{m,y}(x) = \prod_{j : y_j = 1} x_j \prod_{j : y_j = 0} (1 - x_j), \qquad g_a(x) = \sum_y a_y \,\mathrm{ind}_{m,y}(x).$$

Then $g_a(y) = a_y$ on the Boolean cube and $a \mapsto g_a$ is linear. This is the low-degree code: the assignment is represented by the evaluation table of its unique multilinear extension. [16] The Julia function `g_a` constructs the sparse polynomial; block coordinates are carried by `VarLayout` and `VarBlock`; `dependency_blocks` recomputes the actual support dependency.

For five satisfying blocks, let g_i be their multilinear extensions and write $z = (x_1, \ldots, x_5, o, w) \in \mathbb{F}_q^{m'}$. The central analytic object is

$$c_0(z) = F_{\mathrm{arith}}(z) \prod_{i=1}^{5} (g_i(x_i) - o_i).$$

[15] Ground truth: `prop:tseitin-arith-degree` in `ground-truth/gt-10-answer-reduction.tex`.
[16] Ground truth: `sec:ld-encoding` in `ground-truth/gt-03-prelim.tex`.

On a Boolean point, either $F_{\text{arith}} = 0$, or the indicated clause is present and at least one factor $g_i(x_i) - o_i$ vanishes. Hence c_0 vanishes on $\{0,1\}^{m'}$. The code executes the product in `build_c0`; the `BuildC0` certificate replays the structural and actual degree accounts. The implication from a satisfying decoupled instance to Boolean-cube vanishing is checked only on the TB0 fixture and cited in general.[17]

6.2 Division by the Boolean ideal

Let $z_i(1 - z_i) = z_i - z_i^2$. The ground-truth basis proposition says that a degree-bounded polynomial vanishing on the Boolean cube lies in the ideal generated by these m' polynomials. [18] The implementation uses the following deterministic monomial rewrite.

Lemma 6.1 (Executable zero-basis rewrite)**.** *For a coefficient a, a monomial u independent of the displayed power, and $e \geq 2$,*

$$auz_i^e \longmapsto auz_i^{e-1} - auz_i^{e-2}\, z_i(1 - z_i).$$

Iterating in a fixed variable and monomial order yields polynomials c_i and a remainder r, multilinear in every variable, such that

$$c_0 = \sum_{i=1}^{m'} c_i z_i(1 - z_i) + r.$$

If the coordinate degree vector of c_0 is D, then $\deg_j(c_i) \leq D_j$, $\deg_i(c_i) \leq \max(D_i - 2, 0)$, and after reducing coordinates $1, \ldots, i-1$, the running remainder has degree at most one in those coordinates.

Proof. Expanding the right-hand side of one rewrite gives

$$auz_i^{e-1} - auz_i^{e-2}(z_i - z_i^2) = auz_i^e.$$

Thus every step preserves the coefficient polynomial after adding the recorded quotient contribution. It lowers the current exponent to one and places power $e - 2$ in the quotient, proving the degree bounds. Induction over the ordered coordinates gives the displayed decomposition and a final multilinear remainder. If c_0 vanishes on the Boolean cube, then so does r; uniqueness of multilinear interpolation on $\{0,1\}^{m'}$ makes $r = 0$. This is the constructive division used in the proof of `prop:zero-basis` in `ground-truth/gt-10-answer-reduction.tex`. □

`zero_basis_decompose` stores every step as `RewriteStep` inside a `ZeroDecomposition`. `verify_rewrite_step` replays the scalar identity, and `verify_zero_decomposition` reconstructs the entire right side as a normalized coefficient dictionary. Promotion to a zero certificate also requires empty remainder. This is stronger than checking the equality at sampled points, but it remains an instance certificate, not a proof of the PCP soundness theorem.

6.3 PCP view and local verifier

A *PCP view* at z is

$$\text{ev}_z(\Pi) = (g_1(x_1), \ldots, g_5(x_5), c_0(z), c_1(z), \ldots, c_{m'}(z)).$$

[17] Ground truth: `thm:pcp-decider` in `ground-truth/gt-10-answer-reduction.tex`.
[18] Ground truth: `prop:zero-basis` in `ground-truth/gt-10-answer-reduction.tex`.

The proof object is the tuple of evaluation tables for those polynomials. The labels are `def:pcp-proof` and `def:pcp-eval` in `ground-truth/gt-10-answer-reduction.tex`. Julia represents them by `PCPProof` and `PCPView`; `ev_z` refuses a g_i whose actual support escapes block X_i.

The verifier first reconstructs the succinct circuit, Tseitin formula, and arithmetization. It then checks exactly

$$\beta_0 = F_{\mathrm{arith}}(z) \prod_{i=1}^{5} (\alpha_i - o_i), \qquad \beta_0 = \sum_{j=1}^{m'} \beta_j z_j (1 - z_j).$$

These are the formula test and zero-on-subcube test. [19] The Julia function `pcpverifier` executes these two equations on an already supplied `TseitinFormula` and view. In TB2 it does not implement the figure's preceding reconstruction steps; the formula is construction-time data. `build_pcp` combines upstream certificates, coefficient division, degree checking, and stored certified views.

6.4 The four soundness layers

The four layers must not be collapsed.

1. Algebraic soundness under a low-degree premise. Assume every submitted g_i, c_j has individual degree at most d. If the formula-test polynomials are unequal, their difference has total degree at most $(\delta_F + 5d)m'$ under the occurrence-vector account. If the zero-test polynomials are unequal, their difference has total degree at most $(2 + d)m'$. Schwartz–Zippel states that unequal polynomials of total degree at most Δ agree at a uniform point of $\mathbb{F}_q^{m'}$ with probability at most Δ/q. [20] Consequently

$$\Pr[\text{false formula identity}] \le \frac{(\delta_F + 5d)m'}{q}, \qquad \Pr[\text{false zero identity}] \le \frac{(2 + d)m'}{q}.$$

The paper's displayed first numerator uses $2 + 5d$; the structural project obligation substitutes the measured δ_F under the discrepancy stated above. The field size enters nowhere mysteriously: $q = 2^k$ is chosen so both fractions are below the acceptance threshold $1/2$, together with the other PCP parameter obligations. The first structural inequality is extra to the literal `def:pcpparams` predicate in the present project.

If acceptance is greater than $1/2$ and both upper bounds are below $1/2$, each sampled equality is therefore a polynomial identity. The zero identity makes c_0 vanish on the Boolean cube; the formula identity then decodes five Boolean assignments satisfying the decoupled formula, and hence an accepting bounded computation. This conditional theorem is `thm:pcp-decider` in `ground-truth/gt-10-answer-reduction.tex`. Its current project status is `C5`: SKETCH, not CHECKED.

2. Enforcement of the low-degree premise. The simultaneous low-degree game queries points, axis-parallel lines, and diagonal lines. Its classical decider checks consistency of a point value with a claimed line restriction.[21] The theorem extracting approximately consistent low-individual-degree polynomial measurements from a successful quantum strategy is `lem:ld-soundness` in

[19] Ground truth: `fig:pcpverifier` in `ground-truth/gt-10-answer-reduction.tex`.
[20] Ground truth: `lem:schwartz-zippel` in `ground-truth/gt-03-prelim.tex`.
[21] Ground truth: `fig:ld-decider` in `ground-truth/gt-07-ldt.tex`.

`ground-truth/gt-07-ldt.tex`. It is CITED. Running `ld_decider` on honest finite inputs does not prove this extraction theorem.

3. Polynomial identity from random evaluation. This is exactly the Schwartz–Zippel step above, used twice with two different degree bounds. The local program checks evaluations. The analytic theorem turns an acceptance probability into formal identity, provided the degree and field-size premises hold. No sample count can replace that implication.

4. Quantum consistency and rigidity. Answer reduction must connect different players' point and line answers to the same polynomial measurements, control disturbance, and transport the result back to the original verifier. Those operator estimates use low-degree soundness, typed consistency transformations, and later rigidity arguments. They are imported in the soundness contract of `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`; none is a sparse-polynomial identity.

The computer adds exactly this: on a named instance it constructs the polynomials, executes the division, compares normalized coefficients, computes degree and dependency vectors, and evaluates the two local equations. It does not test the theorem. Current statuses reflect that separation: C3 is TESTED, C5 is SKETCH, C1 and C2 remain CONJECTURE, and C8 is TESTED only on its two named circuits.

7 The typed layer: conditionally linear functions

7.1 The inductive object and its laws

Definition 7.1 (Conditionally linear function)**.** Let $V = \mathbb{F}^n$ with fixed coordinate-register subspaces. The unique level-zero conditionally linear (CL) function is zero. A level-ℓ CL function chooses complementary registers $V_1 \oplus V_{>1} = V$, applies a linear map $A : V_1 \to V_1$, and, conditional only on $A(x^{V_1})$, selects a level-$(\ell - 1)$ CL function on $V_{>1}$. Its output is the sum of the first-stage and continuation outputs.

This is `def:cl-func` in `ground-truth/gt-04-cl.tex`. Julia mirrors it with `AbstractCL`, `CLZero`, and `CLStep`. The constructor verifies disjoint factor/rest coordinate sets, a matrix acting inside the factor, a fixed child level, and exact coverage of the rest register. Thus an inhabitant of `CLStep` has a constructed level; linearity is carried by the matrix, not inferred from samples.

The marginal $L_{\le k}$ is the sum of the first k stage outputs, together with their factor spaces and linear maps. This is the decomposition of `lem:cl-kth` in `ground-truth/gt-04-cl.tex`. Julia's `marginal_k` returns a `CLMarginal`; `sum_stage_outputs` recomputes its value.

Lemma 7.2 (Level laws)**.** *Let L be level k, let every continuation R_u be level ℓ, and let L_j act on pairwise disjoint registers at levels ℓ_j. Then*

$$\operatorname{level}(\operatorname{concatenate}(L, R)) = k + \ell, \qquad \operatorname{level}\left(\bigoplus_j L_j\right) = \max_j \ell_j.$$

For CL distributions S_1, S_2, their product, formed by direct-summing the left maps and the right maps on independent seed registers, has level the maximum of the two levels.

Proof. For concatenation, induct on the outer constructor of L. At level zero, grafting R_0 supplies exactly ℓ stages. At a `CLStep`, retain its first linear stage and apply the induction hypothesis to each child; this gives $1 + (k - 1 + \ell) = k + \ell$. This is the construction in `lem:cl-concat` of `ground-truth/gt-04-cl.tex`.

For direct sum, pad each component by zero stages to $r = \max_j \ell_j$. At stage i, use the direct sum of the components' stage matrices on the direct sum of their factor registers. The continuation depends only on the tuple of earlier outputs, hence remains CL. Induction on r proves the law, matching `lem:cl-func-prod` in `ground-truth/gt-04-cl.tex`. A distribution product applies this law separately to its left and right maps, so its level is the same maximum. □

The code functions are `concatenate`, `direct_sum`, `distribution`, and `product`. Their type constructions carry the level laws. The stronger structural hypothesis that all compiler operations of interest close inside one useful algebra is not implied by these individual lemmas.

7.2 Point and line maps

For $V = V_{\mathrm{pt}} \oplus V_{\mathrm{coord}} \oplus V_{\mathrm{dir}}$, write a seed as $(u, s, v) \in \mathbb{F}_q^m \times \mathbb{F}_q \times \mathbb{F}_q^m$. Let L_v^{lnf} be the canonical projection whose kernel is $\mathrm{span}(v)$, so $L_v^{\mathrm{lnf}} u$ is a canonical representative of the affine line $u + \mathbb{F}_q v$. [22] The line-representative use is `def:line-representative` in `ground-truth/gt-07-ldt.tex`. Define $\chi(s)$ by splitting the fixed integer representatives of $\mathbb{F}_q$ into m equal buckets; this requires $m \mid q$. With π_{i-1} zeroing the first $i - 1$ coordinates,

$$
\begin{aligned}
L_{\mathrm{Point}}(u, s, v) &= (u, 0, 0), \\
L_{\mathrm{ALine}}(u, s, v) &= (L^{\mathrm{lnf}}_{e_{\chi(s)}} u, s, 0), \\
L_{\mathrm{DLine}}(u, s, v) &= (L^{\mathrm{lnf}}_{\pi_{\chi(s)-1}(v)} u, s, \pi_{\chi(s)-1}(v)).
\end{aligned}
$$

Their levels are respectively one, two, and three by their stage factorizations.[23] The implementation names are `L_Point`, `L_ALine`, `L_DLine`, `L_lnf`, `chi`, and `pi_prefix`. Since the source's kernel-basis definition does not cover $v = 0$, the code uses $L_0^{\mathrm{lnf}} = I$ and marks the convention SOURCE_REPAIR.

The distribution $\mu_{L,R}$ pushes one uniform shared seed through the two maps.[24] For $(q, m) = (8, 2)$, TB1 enumerates all $8^5 = 32768$ seeds. The axis-line/point histogram has support 512, and the diagonal-line/point histogram has support 18432, including the zero directions. Both agree entry-for-entry with separately transcribed formulas for `lem:alnf` and `lem:dlnf` in `ground-truth/gt-07-ldt.tex`; every marginal is replayed. This finite statement is `C4a`: TESTED. It is not a statistical argument and does not establish quantum low-degree soundness.

7.3 Types and the six-copy sampler

A *typed sampler* is a finite type-graph-indexed family of CL maps sharing one ambient seed space. It first samples an oriented type edge and then pushes one uniform seed through the selected left/right maps. [25] Its induced distribution is `def:typed-sampler-sample` in the same file. Julia's

[22] Ground truth: `def:cl-canonical` in `ground-truth/gt-03-prelim.tex`.
[23] Ground truth: `sec:ld-game` in `ground-truth/gt-07-ldt.tex`.
[24] Ground truth: `def:cl-dist` in `ground-truth/gt-04-cl.tex`.
[25] Ground truth: `def:typed-sampler` in `ground-truth/gt-06-types.tex`.

`TypedSampler` checks exact type coverage, a common field and seed dimension, graph endpoints, and a padded common level.

Detyping encodes the external graph choice into an untyped CL distribution by a rejection-sampling construction. It adds two levels, preserves value-one completeness, and changes soundness error by $16^{|\mathrm{Type}|}$.[26] The Julia function `detype` returns `CitedDetypedVerifier`; the certificate is deliberately CITED. For the 54 answer-reduction types, the displayed factor is 16^{54}, not an executed estimate.

The answer-reduction PCP sampler has the 18 types

$$\{\mathrm{Point}_i, \mathrm{ALine}_i, \mathrm{DLine}_i : 1 \le i \le 6\}$$

and a complete type graph. Copies 1–5 query one m-variate g_i each; copy 6 queries the bundle $(g_1, \ldots, g_5, c_0, \ldots, c_{m'})$ in dimension m'. The six-copy construction and its ambient direct sums are in `sec:ld-compiler` of `ground-truth/gt-10-answer-reduction.tex`. The current code uses `PCPType`, `PCPRegisterLayout`, `PCPCLMap`, `PCPPaddedMap`, and `pcp_sampler`. It carries a visible SOURCE_REPAIR for the copy-6 coordinate-register conflict between the displayed ambient space and the parser table.

Oracularization is the typed transformation in which the isolated roles receive the original left or right CL image while an oracle role receives the common seed and can reconstruct both; the code name is `oracularize_sampler`. Its theorem contract is CITED. [27] The typed answer-reduced sampler is the product of this three-role family with the 18-type PCP family. `typed_sampler_product` produces 54 types and combines levels by a maximum. The code constructs the product and checks its shape, but C4b remains CONJECTURE: the latest TB2 critic held promotion because the lazy `PCPCLMap`'s stage depth was not itself enough to make the full CL decomposition a constructor-level fact. Local certificate labels must not be confused with a promoted claim.

The typed decider implements the five guarded checks of `fig:decider-pcp`: global consistency, input consistency, input low degree, proof encoding (individual and simultaneous low degree), and the PCP game check.[28] The Julia names are `TypedAnswerReducedDecider`, `typed_answer_reduced_decider`, `LDParams`, `ld_decider`, and `PCPGameCall`. `answer_reduce_pcp` constructs the typed verifier of level $\max(\ell, 3)$ on its fixture and attaches explicit assumptions.

The full theorem contract is conditional:

ASSUME: V is an ℓ-level normal-form verifier,
$T(n) = (2^{\lambda n})^{\mu}, \quad Q(n) = (\lambda n)^{\mu}$,
$\mathrm{TIME}_D(n) \le T(n), \quad \mathrm{TIME}_S(n) \le Q(n)$;

PROVE: $\mathsf{AnswerReduce}(V)$ has level $\max\{\ell + 2, 5\}$,
the stated polynomial runtimes and directed completeness,
soundness, and entanglement implications.

This is `thm:ar` in `ground-truth/gt-10-answer-reduction.tex`. The finite guarded decider is executable; the quantum implication, detyping, and asymptotic theorem are CITED.

[26] Ground truth: `lem:detyping-verifiers` in `ground-truth/gt-06-types.tex`.
[27] Ground truth: `sec:orac-def` in `ground-truth/gt-09-oracularization.tex`.
[28] Ground truth: `fig:decider-pcp` in `ground-truth/gt-10-answer-reduction.tex`.

7.4 The structural hypothesis

Introspection is the cited verifier transformation that makes encoded prover-side measurement information available to a smaller verifier while preserving a directed soundness contract. [29] *Anchoring* inserts special anchor questions with positive probability; *parallel repetition* runs several copies with an AND acceptance condition against possibly correlated strategies. Their combined repetition contract is cited from `thm:repetition` in `ground-truth/gt-11-parallel-repetition.tex`.

The relevant closure equations are

$$\mathsf{Introspect}(\mathcal{Q}) \subseteq \mathcal{Q}, \qquad \mathsf{PCPCompose}(\mathcal{Q}) \subseteq \mathcal{Q}, \qquad \mathcal{Q} \times \mathcal{Q} \subseteq \mathcal{Q}.$$

Concatenation, direct sum, the shared-seed distribution, and product are visibly CL-preserving in the base inductive datatype. Affine restriction is linear after earlier stages choose a direction, and random-point identity tests are pushforwards of a shared uniform seed. Typed graph choice is external, and the cited detyping transformation moves it back into two further CL levels.

This does not characterize every admissible query distribution, prove that a generic PCPP can be expressed in the algebra, or prove closure under the full introspection compiler. The structural statement is exactly `C7`: CONJECTURE. Introspection soundness and normal-form closure are not implemented. The present datatype makes several local closure laws legible, not the global compression theorem.

8 Correspondence tables

The evidence grade describes the local certificate leaf. The claim column is the current status in `claims/CLAIMS.md`; it can remain weaker than a certificate label when an adversarial verdict has not promoted the claim. CONSTRUCTED means enforced by a constructor, CHECKED means deterministic replay against an attached object, and CITED means imported from the named ground-truth statement.

8.1 Description front end (planned TB3)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
`sec:tms` `gt-03-prelim.tex`	Machine conventions in Definition 1.1; costed bridge in Theorems 3.1–3.2	`Quoted{A}`	`Quote`, `Eval`, `Specialize`	CITED; absent	none
`def:decider` `gt-05-games-normalform.tex`	Five-input format in Definition 1.2; bounds preserved by Theorem 3.3	`Quoted{Decider}`	`description_size`	CITED; absent	none
`DESIGN` §1.1	Determinism, fuel, quotation, and specialization: Theorems 2.2–2.4 and Lemma 2.5	`ClosedProgram`, `Quoted`	`Eval`, `specialize`	analytic only; absent	none

[29] Ground truth: `thm:introspection` in `ground-truth/gt-08-introspection.tex`.

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
prop: standard-succinct-sat gt-10-answer-reduction.tex	Transition locality (Lemma 5.1) and succinct 3SAT (Theorem 5.2)	BoundedTrace, Succinct3SAT	bounded_trace, cook_levin	CITED; absent	none
sec:succinct-deciders gt-10-answer-reduction.tex	Three shared reads become five independent blocks	SuccinctDecoupled5SAT	decouple5	CITED; absent	none

8.2 Polynomial rung (TB0)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:tseitin gt-10-answer-reduction.tex	Circuit becomes a linear-size formula with auxiliary wires	Circuit, TseitinFormula	tseitin, occurrences	CHECKED occurrence	C2 CONJ.
def: formula-arithmetization gt-10-answer-reduction.tex	Boolean connectives become a formal polynomial	Poly	arith_q, actual_degrees	CHECKED	C8 TESTED
sec:ld-encoding gt-03-prelim.tex	Assignment becomes its multilinear extension	Poly	g_a, dependency_blocks	CONSTRUCTED/ CHECKED	C3 TESTED
prop:zero-basis gt-10-answer-reduction.tex	Divide by $z_i(1-z_i)$ and require zero remainder	ZeroDecomposition	zero_basis_decompose, verify_zero_ decomposition	CHECKED	C2 CONJ.
fig:pcpverifier gt-10-answer-reduction.tex	Check formula and zero identities at one view	PCPView	pcpverifier	CHECKED finite views	C1 CONJ.
thm:pcp-decider gt-10-answer-reduction.tex	Acceptance $> 1/2$ decodes an accepting trace, assuming low degree	PCPProof	build_pcp, parameter_policy	CITED/ CHECKED parts	C5 SKETCH

8.3 CL and low-degree-test rung (TB1)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
def:cl-func gt-04-cl.tex	CL functions are inductive conditional linear stages	AbstractCL	CLZero, CLStep, level	CONSTRUCTED	C4a TESTED
lem:cl-kth gt-04-cl.tex	A marginal is the sum of the first k stages	CLMarginal	marginal_k, sum_stage_outputs	CHECKED	C4a TESTED
lem:cl-concat, lem:cl-func-prod gt-04-cl.tex	Concatenation adds levels; direct sum takes a maximum	AbstractCL	concatenate, direct_sum, product	CONSTRUCTED	C7 CONJ.
lem:alnf, lem:dlnf gt-07-ldt.tex	CL pushforwards equal the line/point distributions	CLDistribution	L_Point, L_ALine, L_DLine, histogram	CHECKED TB1	C4a TESTED
fig:ld-decider gt-07-ldt.tex	Point/line answer consistency and degree format	LDParams	ld_decider, restrict	CHECKED fixture	C4a TESTED
lem:ld-soundness gt-07-ldt.tex	Successful quantum strategy yields polynomial measurements	theorem leaf	no proof checker	CITED	C5 SKETCH

8.4 Typed answer-reduction rung (TB2)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
`def:typed-sampler` `gt-06-types.tex`	A type graph selects a pair of CL maps sharing one seed	`TypedSampler`	`TypedSampler`, `sample`, `pad_level`	CONSTRUCTED base	C4b CONJ.
`sec:ld-compiler` `gt-10-answer-reduction.tex`	Six low-degree copies form an 18-type family	`PCPCLMap`	`pcp_sampler`, `intrinsic_pcp_levels`	local CHECKED; held	C4b CONJ.
`sec:ld-compiler` `gt-10-answer-reduction.tex`	Product with the oracularized sampler has 54 types	`TypedProductCL`	`oracularize_sampler`, `typed_sampler_product`	CHECKED shape; theorem cited	C4b CONJ.
`fig:decider-pcp` `gt-10-answer-reduction.tex`	Five guarded checks connect input, proof, low degree, and PCP	`TypedAnswer` `ReducedDecider`	`typed_answer_reduced_decider`	CHECKED finite paths	no current row
`lem:detyping-verifiers` `gt-06-types.tex`	Remove types at cost +2 levels and 16^{54} soundness factor	`CitedDetyped` `Verifier`	`detype`	CITED	C4b CONJ.
`thm:ar` `gt-10-answer-reduction.tex`	Conditional complexity, completeness, soundness, entanglement contract	`TypedAnswer` `ReducedVerifier`	`answer_reduce_pcp`, `AnswerReduce`	CHECKED/ CITED	C5 SKETCH

8.5 Midpoint diagnostic (TB0.5)

Analytic anchor	Analytic statement	IR sort	Implementation	Grade	Claim
`handoff.md` midpoint diagnostic	False level-n claim has value $1 - 2^{-n}$ under the stated orbit condition	`Test/Ask/Coin` terms	`toys/midpoint`	proof plus exact tests	C6 PROVED
`toys/midpoint/PROOF.md`	Sequential r-copy AND value is $(1 - 2^{-n})^r$; constant gap costs $\Theta(2^n)$ copies	exact dynamic program	`sequential_value`	proof plus exact tests	N1 PROVED

8.6 Compression and fixed point (planned TB4)

Paper object	Analytic statement	IR sort	Julia name	Grade	Claim
`fig:compress` `gt-12-compression.tex`	Repeat ∘ AnswerReduce ∘ Introspect	`ClosedProgram` `Compressor`	`Compress`	CITED; absent	C7 CONJ.
`thm:compression` `gt-12-compression.tex`	Nine-level gap-preserving compression with dimension bound	contract tree	no current implementation	CITED	C7 CONJ.
`fig:halt_f` `gt-12-compression.tex`	Self-application, Kleene, and YCode coincide by Theorems 4.2–4.4	`PartialProgram`, `Quoted`	`Hole`, `Specialize`, `Fix/YCode`	CITED; absent	none

9 What is analytic and what is computed

9.1 The evidence boundary

The analytic content is the sequence of transformations and the implications attached to them. The computer currently realizes a strict middle segment.

Class	Present content	What would be required to move the boundary
Executed constructions	Circuit/Tseitin syntax; formula arithmetization; multilinear extensions; c_0; zero-basis quotients; point/line maps; typed products; guarded TB2 decider	Scale-independent front end for quoted programs and a constructor-faithful lazy CL stage representation for the PCP sampler
Certified identities	Normalized coefficient equality; zero remainder; structural and actual degrees; dependency blocks; CL marginals; TB1 exact histograms; finite parser and branch checks	General semantic certificates connecting trace, SAT, Tseitin, and arithmetization, plus promotion through adversarial verdicts
Cited analytic leaves	Cook–Levin asymptotics; general Tseitin semantics; PCP soundness; low-degree extraction; oracularization; detyping; introspection; rigidity; anchored repetition; compression	Formal proofs over quantified machines, distributions, Hilbert spaces, measurements, approximation metrics, and asymptotic bounds—not more finite samples

Lean or another proof assistant could check the algebraic core once finite fields, sparse polynomials, interpolation, and polynomial division are formalized. That would move the general zero-basis identity and degree lemmas from prose to proof terms. It would not by itself certify `lem:ld-soundness`: that leaf requires a library for finite-dimensional Hilbert spaces, POVMs, state-dependent distances, tensor-code tests, and the quantitative extraction argument. Rigidity and oracularization require the same operator layer. Detyping additionally needs a formal probability and rejection-sampling argument with the $16^{|\mathrm{Type}|}$ loss.

Anchoring and parallel repetition are combined in Repeat and are theorem-backed rather than executed here. Introspection and compression would require a still broader object: a formal semantics for quoted verifier descriptions, resource bounds under specialization and universal evaluation, and a proof that the directed value and entanglement implications compose. The relevant final statements are `fig:compress` and `thm:compression` in `ground-truth/gt-12-compression.tex`.

9.2 The midpoint diagnostic

The recursive midpoint verifier for $y = f^{2^n}(x)$ asks for z and checks one of

$$z = f^{2^{n-1}}(x), \qquad y = f^{2^{n-1}}(z)$$

with a fresh fair coin. It has a compact fixed-point description and perfect completeness. Under the orbit-prefix condition stated in C6, the exact optimum for a false claim is nevertheless

$$p_n = 1 - 2^{-n}.$$

Claim C6 is PROVED. For adaptive sequential AND repetition, N1 is also PROVED and gives value $(1 - 2^{-n})^r$; reaching error at most $1/2$ requires $r = \Theta(2^n)$, hence $\Theta(n2^n)$ transcript cost in the stated unit-cost model. Neither claim concerns quantum parallel repetition or the actual Compress operator.

The diagnostic isolates the point of the whole construction. A recursive term can represent an exponentially long computation, and a fixed point can make that term self-referential, while the acceptance gap collapses too quickly for iteration. Representability is easy; an iteratable, gap-preserving analytic transformation is the hard requirement.

9.3 Final accounting

The lambda reformulation removes none of the mathematically difficult leaves. It makes quotation, hardwiring, description size, timeout, and self-reference explicit in one syntax. The polynomial IR makes the Cook–Levin-to-PCP middle visible as actual algebra, and the CL datatype exposes several sampler closure laws. What remains unchanged is the proof that high success in the typed quantum game yields consistent low-degree polynomial measurements, survives oracularization and detyping with the stated quantitative loss, survives introspection and anchored parallel repetition, and composes into the gap-preserving compression theorem. Those are analytic theorems, presently CITED; they are neither consequences of homoiconicity nor conclusions of the finite computations reported here.